Denoising Autoencoder

Deep Learning | Implementation and interview guide

Purpose and intended demonstration

Train an encoder-decoder neural network to reconstruct clean signals from noisy observations.

How the implementation works

A 16-dimensional input passes through a five-unit tanh bottleneck and linear decoder. Backpropagate mean squared reconstruction loss to both layers. Fresh Gaussian corruption is sampled each epoch.

Evaluation and evidence

Train and test on independent sinusoidal signals. Compare reconstruction MSE with noisy-input MSE. Store clean, noisy and reconstructed examples for inspection.

Denoising Autoencoder

Deep Learning | Implementation and interview guide

Setup and commands

Requires Python 3.10 or newer and NumPy. Create a virtual environment: python -m venv .venv. On Windows use .venv\Scripts\python.exe if activation is unavailable. Install with python -m pip install -r requirements.txt. Run python app.py, then open results/report.html in your browser. Run python -m unittest discover -v to execute tests. The default seed is 42; change it with --seed. Replace sample_input.json using --input path-to-file.json, preserving its schema. Algorithm-only projects generate synthetic arrays from the seed rather than consuming external files.

Architecture and file responsibilities

CLI arguments -> JSON input or seeded generator -> domain pipeline -> validation or evaluation -> JSON and HTML artifacts. Imported functions can be used in a notebook or a separate application. Model checkpoints preserve learned arrays where applicable; see app.py for parameter order. This package does not include a production web service. Each project is independent and includes its own helper code so the ZIP can be extracted and run alone. app.py holds the experiment and domain functions: dataset, init, forward, loss_grad, train, run. kit.py contains reporting, input loading and only the retrieval or generation helpers needed by this project. sample_input.json contains editable inputs. test_app.py checks domain behaviors; test_reporting.py checks metrics and HTML output boundaries. results/metrics.json records outputs and results/report.html provides a browser-readable report. Numeric projects also save arrays or learned parameters to results/artifacts.npz; inspect its keys with numpy.load. MODEL_CARD.md explains intended use and limits. .github/workflows/tests.yml runs tests and the fixture experiment. No API key is needed.

Limitations and next improvements

The data lies on a simple low-dimensional manifold; this is signal denoising, not a diffusion image generator. Add held-out frequencies, realistic noise, validation-based early stopping and reconstruction plots. Compare a linear PCA denoiser.

Interview walkthrough

Present the problem and demonstrate one normal example, then one failure or boundary case. Explain the baseline or fixture mode, the split or validation design, and one limitation before discussing an extension. Be ready to trace data through the functions listed above. Explain why the chosen metric matches the task and why the available results cannot establish production performance. Add your own documented experiment and be precise about what you implemented or changed. These are AI-assisted educational projects; filenames and synthetic scores are not evidence of client work.

Verified build results

project

"Denoising Autoencoder"

noisy mse

0.12138432080543668

denoised mse

0.029083519011437645

initial training loss

0.3781584568044373

final training loss

0.028254579514999406

bottleneck

5

See results/metrics.json for all observations and TEST_RESULTS.txt for the executed tests. All datasets and replay responses shipped here are synthetic. Results can vary across seeds and numerical library versions.